INSTITUTO UNIVERSITARIO ADVENTISTA DE VENEZUELA

Manual de instalación local

Cómo descargar el Sistema de Servicio Comunitario, preparar la base de datos y ejecutarlo en su computadora para revisarlo y probarlo.

> **Para quién es este manual** Para el tutor o evaluador que va a revisar el sistema. No hace falta saber programar ni instalar PostgreSQL: el proyecto trae su propia base de datos. Solo hay que descargar, hacer doble clic en dos archivos y abrir el navegador.

Versión del manual: 1.0

Dirigido a: tutor / evaluador del proyecto

Tiempo de instalación: entre 5 y 15 minutos la primera vez

Sistema operativo: Windows 10 u 11 (64 bits)

Repositorio: github.com/JesusQuilleli/servicio-comunitario-iunav

Contenido

1. **1.** Qué va a instalar y cómo funciona
2. **2.** Requisitos
3. **3.** Paso 1: descargar el proyecto
4. **4.** Paso 2: instalar (una sola vez)
5. **5.** Paso 3: iniciar el sistema
6. **6.** Paso 4: verificar que todo esté bien
7. **7.** Paso 5: detener el sistema
8. **8.** Las cuentas de prueba y qué revisar con cada rol
9. **9.** La base de datos: cómo se crea, cómo se reinicia y cómo se respalda
10. **10.** Si algo falla
11. **11.** Opción B: instalación manual (otro sistema operativo o si el kit falla)
12. **12.** Cómo desinstalar y dejar el equipo limpio
13. **13.** Anexo: estructura y comandos

> **Resumen en una frase**
>
> Descargue el proyecto, haga doble clic en `local\instalar.cmd` una sola vez, y después abra el sistema con `local\iniciar.cmd` . La base de datos se crea y se llena sola.

1. Qué va a instalar y cómo funciona

El proyecto tiene dos partes que deben estar encendidas al mismo tiempo:

Parte	Qué es	Qué hace
La web	La aplicación que usted ve	Pantallas de inicio de sesión, panel, bitácoras, validación, reportes. Hecha con Next.js.
La API	El servidor de datos	Recibe lo que hace la web, aplica las reglas (por ejemplo, que solo el tutor valide) y guarda todo. Hecha con Express.
La base de datos	Donde se guarda todo	PostgreSQL. **Viene incluida dentro del proyecto**: no hay que instalarla por separado.

Qué significa "local"

Todo se ejecuta **dentro de su computadora**. No se conecta a internet, no toca ningún servidor externo y no necesita claves de terceros. La única vez que se usa internet es durante la instalación, para descargar las dependencias del proyecto.

Las direcciones que se usan

Servicio	Dirección	Para qué
Web (la aplicación)	`http://localhost:3000`	Es la que se abre en el navegador.
API (los datos)	`http://localhost:4000`	La usa la web por dentro. También sirve para comprobar que está viva.
Base de datos	`localhost:55432`	Solo la usa la API. No se abre en el navegador.

> **Por qué la base usa el puerto 55432 y no el 5432**
>
> El 5432 es el puerto normal de PostgreSQL. Se usa el 55432 a propósito para no chocar con un PostgreSQL que usted ya tenga instalado. Si lo tiene, no pasa nada: este proyecto usa el suyo propio y no lo toca.

Qué va a encontrar

- **4 tipos de usuario:** administrador, coordinador, tutor y estudiante.
- **8 cuentas de prueba ya creadas** (4 roles, con varios estudiantes y tutores).
- **Datos de ejemplo:** un período académico, grupos, bitácoras y servicios ya registrados.
- **Reportes** en PDF y Excel, y documentos como actas y certificados.

> **No hay que crear la base de datos a mano**
>
> El instalador crea la base, las 9 tablas y los datos de ejemplo automáticamente. Usted no escribe ni una línea de SQL.

2. Requisitos

Requisito	Versión	Cómo comprobarlo
Windows	10 u 11, 64 bits	—
Node.js	20.9 o superior (se recomienda 22 LTS)	Abra una terminal y escriba `node -v`. Debe mostrar `v22…` o superior.
Git	Cualquier versión reciente	Escriba `git --version`. Si no lo tiene, puede usar el ZIP (ver paso 1).
Espacio en disco	Unos 4 GB libres	Las dependencias y la base local ocupan alrededor de 2 GB.
Internet	Solo la primera vez	Se descargan las dependencias y PostgreSQL (unos 200 MB).

Si no tiene Node.js

La forma más rápida es con **winget** (viene con Windows 10 y 11). Abra una terminal y ejecute:

```
winget install OpenJS.NodeJS.LTS
```

Si no tiene winget, descárguelo de **https://nodejs.org/es/download** (elija la versión *LTS*) e instálelo con las opciones por defecto. Después **cierre y vuelva a abrir** la terminal para que Windows reconozca el comando.

> **Windows puede pedir permiso**
>
> Al ejecutar los archivos `.cmd`, Windows puede mostrar un aviso de seguridad. Elija *"Más información"* → *"Ejecutar de todas formas"*. Son archivos de texto que puede abrir con el Bloc de notas para revisarlos antes de ejecutarlos.

> **Antivirus**
>
> Algunos antivirus corporativos bloquean programas que "abren un servidor local". Si el sistema no arranca y no hay un error claro, agregue una excepción para la carpeta del proyecto o consulte con el área de sistemas.

3. Paso 1: descargar el proyecto

Elija **una** de las dos formas. La primera es la recomendada.

Forma A — con Git (recomendada)

1. Abra una terminal (menú Inicio → escriba `PowerShell` → Enter).
2. Vaya a la carpeta donde quiere guardar el proyecto. Por ejemplo:

```
cd $HOME\Documents
```

3. Descargue el proyecto:

```
git clone https://github.com/JesusQuilleli/servicio-comunitario-iunav.git
```

4. Entre a la carpeta descargada:

```
cd servicio-comunitario-iunav
```

Forma B — con el ZIP (si no tiene Git)

1. Abra en el navegador:
   `https://github.com/JesusQuilleli/servicio-comunitario-iunav`
2. Pulse el botón verde **"Code"** y luego **"Download ZIP"**.
3. Descomprima el archivo. Windows crea una carpeta como **`servicio-comunitario-iunav-master`**.
4. Abra esa carpeta. Dentro debe ver las carpetas **`local`**, **`server`**, **`web`** y **`docs`**.

Cómo saber que la descarga quedó bien

La carpeta principal debe tener esta forma:

```
servicio-comunitario-iunav\
├── local\        ← el kit de instalación (aquí están los .cmd)
├── server\       ← la API
├── web\          ← la web
├── deploy\       ← scripts para publicarlo en un servidor (no se usan aquí)
├── docs\         ← este manual y otras guías
└── HANDOFF.md    ← ficha técnica del proyecto
```

> **Error frecuente: descomprimir "un nivel de más"**
>
> A veces el ZIP crea una carpeta dentro de otra con el mismo nombre. Si al entrar a **`local`** no ve los archivos `instalar.cmd`, `iniciar.cmd` y `detener.cmd`, retroceda un nivel: está en la carpeta equivocada.

4. Paso 2: instalar (una sola vez)

Este paso se hace **una única vez**. Prepara las dependencias, la base de datos y los datos de ejemplo.

> **Haga doble clic en `local\instalar.cmd`**
>
> Se abre una ventana negra que va mostrando el progreso. No la cierre.

Qué va a ver

```
============================================================
   Servicio Comunitario IUNAV
   Instalacion local (una sola vez)
============================================================

Node.js 22.x.x detectado.
[1/4] Revisando las dependencias del proyecto...
    - local: instalando dependencias. Puede tardar varios minutos...
    - server: instalando dependencias. Puede tardar varios minutos...
    - web: instalando dependencias. Puede tardar varios minutos...

[2/4] Preparando la base de datos local...
Primera vez: se prepara la base de datos local. Puede tardar un minuto...
Base de datos local preparada.
Base de datos local lista en el puerto 55432.

[3/4] Creando las tablas y los datos de ejemplo...
Esquema aplicado correctamente.
  + creado cmendoza@iunav.edu.ve (admin)
  ...
Seed completado.

[4/4] Instalacion terminada.

============================================================
   Listo. Para usar el sistema:

     1. Abre la carpeta "local" y haz doble clic en
        "iniciar.cmd".
     2. Se abrira el navegador con el sistema ya en marcha.
     3. Para cerrar todo, haz doble clic en "detener.cmd".

   Cuentas de prueba:
     Administrador  cmendoza@iunav.edu.ve   admin123
     Coordinador    mlopez@iunav.edu.ve     coord123
     Tutor          jperez@iunav.edu.ve     tutor123
     Estudiante     gmartinez@iunav.edu.ve  est123
============================================================
```

Cuánto tarda

Entre **5 y 15 minutos**, según la velocidad de su conexión. La mayor parte del tiempo se va descargando las dependencias y PostgreSQL.

> **Es normal que parezca “congelado”**
>
> Durante la descarga, la ventana puede quedarse quieta varios minutos. No la cierre: está trabajando. Al final siempre aparece la palabra **“Listo”**.

> **Si la instalación falla**
>
> La ventana se queda abierta y muestra un mensaje con la causa. Anótelo (o tome una foto de la pantalla) y revise la sección **“10. Si algo falla”**. Lo más común es que no haya internet o que Node.js sea una versión vieja.

Qué acaba de pasar por dentro

Se creó	Para qué
`local\.pgdata\`	La base de datos completa (su información vive aquí).
`local\.env.local`	La contraseña de esa base y el secreto de las sesiones. Se genera una sola vez.
`web\.env.local`	Le dice a la web dónde está la API.
`server\sql\schema.sql`	Ya venía en el proyecto: define las 9 tablas.

5. Paso 3: iniciar el sistema

> **Haga doble clic en `local\iniciar.cmd`**
>
> Levanta la base de datos, la API y la web, y abre el navegador automáticamente.

Qué va a ver

```
============================================================
   Servicio Comunitario IUNAV
   Inicio local
============================================================

Node.js 22.x.x detectado.
[1/4] Levantando la base de datos local...
Base de datos local lista en el puerto 55432.

[2/4] Actualizando el esquema y los datos de ejemplo...
        (es seguro repetirlo: no borra lo que ya hayas creado)
Esquema aplicado correctamente.
Seed completado.

[3/4] Iniciando la API y la web...
        La primera vez puede tardar un minuto: la web compila.
  - Iniciando la API...
  - Iniciando la web (la primera vez compila y tarda mas)...
  - Esperando a que la API conecte con la base de datos...
    API lista.
  - Esperando a que la web responda...
    Web lista.

  Abriendo el navegador...

  Sistema en marcha: http://localhost:3000

[4/4] Abriendo el navegador...

============================================================
   Sistema en marcha

     Web:  http://localhost:3000
     API:  http://localhost:4000/api/health

   Cuentas de prueba:
     Administrador  cmendoza@iunav.edu.ve   admin123
     Coordinador    mlopez@iunav.edu.ve     coord123
     Tutor          jperez@iunav.edu.ve     tutor123
     Estudiante     gmartinez@iunav.edu.ve  est123

   Para cerrarlo todo: doble clic en detener.cmd
============================================================
```

Al final se abre el navegador en **http://localhost:3000** con la pantalla de inicio de sesión.

La API y la web trabajan en segundo plano

A diferencia de la instalación, aquí
no
se abren dos ventanas negras. Los servicios corren ocultos y dejan su registro en **`local\logs\`**. Eso es correcto: la ventana del `iniciar.cmd` se puede cerrar después de que diga “Sistema en marcha”.

La primera vez tarda más

La web compila al arrancar. Puede tardar entre 20 y 60 segundos. En los arranques siguientes es mucho más rápido (unos 10 segundos).

Qué hacer si el navegador no se abre solo

Abra el navegador y escriba la dirección a mano:

```
http://localhost:3000
```

6. Paso 4: verificar que todo esté bien

Haga estas dos comprobaciones. Toman menos de un minuto.

Comprobación 1: la API y la base están conectadas

En el navegador abra:

```
http://localhost:4000/api/health
```

Debe ver un texto parecido a este:

```
{"ok":true,"baseDeDatos":true,"servicio":"IUNAV Servicio Comunitario API","hora":"..."}
```

Qué ve	Qué significa
`"ok":true,"baseDeDatos":true`	**BIEN** Todo está conectado.
`"ok":false` o `"baseDeDatos":false`	**MAL** La API está viva pero no llega a la base. Ejecute `detener.cmd` y vuelva a `iniciar.cmd`.
La página no carga	**MAL** La API no está corriendo. Vuelva a ejecutar `iniciar.cmd`.

Comprobación 2: se puede entrar con cada rol

Abra `http://localhost:3000` y pruebe las cuatro cuentas de la sección 8. Cada una debe entrar a un panel distinto, con un menú lateral diferente.

> **La pantalla de inicio de sesión muestra las cuentas**
>
> En la parte inferior del formulario aparecen las cuentas de demostración. Puede pulsarlas para que se rellenen solas.

> **Si después de varios intentos fallidos el sistema no lo deja entrar**
>
> Por seguridad, el inicio de sesión permite 10 intentos cada 15 minutos. Si se pasa, espere un rato o reinicie con `detener.cmd` y `iniciar.cmd`.

7. Paso 5: detener el sistema

> **Haga doble clic en `local\detener.cmd`**
>
> Cierra la web, la API y la base de datos, en ese orden.

```
============================================================
   Servicio Comunitario IUNAV
   Detener el sistema local
============================================================

[1/2] Cerrando la API y la web...
  - Cerrando la API (pid 1234)...
  - Cerrando la web (pid 5678)...

[2/2] Cerrando la base de datos local...
Cerrando la base de datos local...
Base de datos local detenida.

============================================================
   Todo cerrado. Ya puedes apagar el equipo.
============================================================
```

> **Es seguro hacerlo siempre**
>
> Los datos que haya creado se conservan. La próxima vez que abra `iniciar.cmd` todo sigue donde lo dejó.

Si intenta iniciar dos veces seguidas sin detener, el sistema se da cuenta y le dice que ya está en marcha, en lugar de fallar.

8. Las cuentas de prueba y qué revisar con cada rol

Rol	Correo	Contraseña	Qué puede hacer
Administrador	`cmendoza@iunav.edu.ve`	`admin123`	Todo: usuarios, configuración, períodos, reportes.
Coordinador	`mlopez@iunav.edu.ve`	`coord123`	Usuarios, grupos, configuración y reportes.
Tutor	`jperez@iunav.edu.ve`	`tutor123`	Validar bitácoras y servicios, ver sus grupos y reportes.
Estudiante	`gmartinez@iunav.edu.ve`	`est123`	Registrar sus horas (bitácora y servicio) y ver su avance.

Además existen **otras cuentas** con los mismos roles, para probar grupos con varios integrantes: `ltorres@iunav.edu.ve` (tutor), `dhernandez@iunav.edu.ve`, `vgarcia@iunav.edu.ve` y `ssanchez@iunav.edu.ve` (estudiantes, contraseña `est123`).

Guion sugerido de revisión

Si quiere comprobar que las reglas del sistema se cumplen, este recorrido toca lo más importante en unos 10 minutos:

Con la cuenta	Haga esto	Qué debe pasar
Estudiante	Registrar una bitácora con una foto de evidencia.	Queda en estado *Pendiente* y suma 0 horas al avance.
Estudiante	Intentar entrar a "Validación" escribiendo la dirección a mano (`/validacion`).	El sistema lo devuelve a su panel: no tiene permiso.
Tutor	Abrir "Validación" y aprobar la bitácora recién creada.	El registro pasa a *Aprobado* y suma a las horas del estudiante.
Tutor	Registrar horas o crear usuarios.	No puede: esas opciones no aparecen en su menú.
Coordinador	Crear un grupo y asignarle tutor y estudiantes.	El grupo aparece en el panel del tutor asignado.
Administrador	Generar el consolidado en PDF y en Excel desde "Reportes".	Se descarga el archivo con las horas del período.

> **Cada rol tiene su propio menú**
>
> Si una opción no aparece, no es un error: es que ese rol no tiene permiso para ella. El sistema lo controla en el servidor, no solo en la pantalla.

9. La base de datos: cómo se crea, cómo se reinicia y cómo se respalda

No hay que pasarle ninguna base de datos

El proyecto **crea la base solo**. El archivo **`server\sql\schema.sql`** define las 9 tablas y el instalador lo aplica. Después, el *seed* llena esa base con los datos de ejemplo (usuarios, período, grupos, bitácoras).

Las 9 tablas que se crean

Tabla	Qué guarda
`usuarios`	Las personas y su rol (admin, coordinador, tutor, estudiante).
`periodos_academicos`	Los períodos (por ejemplo, 2026-I).
`grupos_proyectos`	Los proyectos comunitarios y quién los tutoriza.
`integrantes_grupo`	Qué estudiantes pertenecen a cada grupo.
`bitacoras_comunitario`	Las horas registradas y su validación.
`control_servicios_institucionales`	Las horas de servicio social o ambiental.
`configuracion_sistema`	Las metas de horas (90 directas y 30 de impacto).
`plantillas_documento`	Los documentos modelo (bitácora, acta, informe, certificado).
`solicitudes_cambio`	Los cambios de datos de cuenta que requieren autorización.

Sus datos se conservan

Todo lo que usted cree queda guardado en **`local\.pgdata\`**. Al cerrar y volver a abrir, sigue ahí. El *seed* es inteligente: si un usuario o un grupo ya existe, no lo vuelve a crear ni lo borra.

Cómo empezar de cero

Si quiere volver al estado inicial (solo las cuentas y datos de ejemplo), ejecute desde **`server\`**:

```
npm run db:wipe
```

Eso limpia los datos y deja un administrador con la contraseña `admin123`. **Atención:** borra todo lo que haya cargado.

Cómo respaldar y restaurar

Lo más simple es copiar la carpeta completa mientras el sistema está detenido:

1. Ejecute `detener.cmd`.
2. Copie la carpeta **`local\.pgdata\`** a un lugar seguro.
3. Para restaurar, pegue esa carpeta en el mismo lugar y abra `iniciar.cmd`.

> **No borre `local\.env.local`**
>
> Ese archivo guarda la contraseña con la que se creó la base. Si lo borra pero conserva **`local\.pgdata\`**, la base queda inaccesible. Si le pasa, tendrá que borrar también **`.pgdata\`** y empezar de cero.

10. Si algo falla

Antes de nada: **ejecute `detener.cmd` y luego `iniciar.cmd`**. Eso resuelve la mayoría de los casos. Si no, busque su síntoma en esta tabla.

Lo que ve	Causa probable	Solución
"No se encontro Node.js en este equipo"	Node.js no está instalado o la terminal es vieja.	Instale con `winget install OpenJS.NodeJS.LTS`, cierre la ventana y vuelva a abrir el `.cmd`.
"La version de Node.js instalada es la ... y hace falta la 20.9"	Node.js viejo.	Actualice con `winget upgrade OpenJS.NodeJS.LTS`.
"Cannot find module 'embedded-postgres'"	La instalación no terminó de bajar las dependencias de **`local\`**.	Vuelva a ejecutar `instalar.cmd` con internet.
"el puerto 4000 ya esta ocupado" (o 3000)	Quedó una ejecución anterior viva o hay otro programa en ese puerto.	Ejecute `detener.cmd`. Si sigue, cierre el otro programa o reinicie el equipo.
"no se pudo conectar a PostgreSQL" / `"baseDeDatos":false`	La base no arrancó o se cerró.	`detener.cmd` y `iniciar.cmd`. Si persiste, revise **`local\logs\postgres.log`**.
La API no arranca y el log dice algo de `admin` o contraseña	Se borró o editó **`local\.env.local`**.	Borre **`local\.pgdata\`** y **`local\.env.local`**, y ejecute `instalar.cmd` otra vez.
La web muestra una pantalla en blanco o un error de compilación	Caché de Next.js dañada.	Detenga todo, borre la carpeta **`web\.next\`** y vuelva a `iniciar.cmd`.
"Too many requests" al iniciar sesión	Más de 10 intentos en 15 minutos.	Espere 15 minutos o reinicie con `detener.cmd` + `iniciar.cmd`.
La instalación se queda "congelada"	Está descargando dependencias (200 MB o más).	Espere. No cierre la ventana. Verifique que tenga internet.
El antivirus impide arrancar la base	El antivirus bloquea procesos que escuchan en un puerto.	Agregue una excepción para la carpeta del proyecto.
Windows no deja ejecutar el `.cmd`	Aviso de seguridad.	*Más información → Ejecutar de todas formas.*

Qué enviar si pide ayuda

Adjunte estos datos; con ellos se puede diagnosticar casi cualquier fallo:

- Una foto o captura de la ventana con el error.

- El archivo **`local\logs\api.log`**.
- El archivo **`local\logs\web.log`**.
- El archivo **`local\logs\postgres.log`**.
- La salida de `node -v`.

> **Los registros no contienen contraseñas**
>
> Son texto de diagnóstico. Aun así, revíselos antes de compartirlos.

11. Opción B: instalación manual

Use esta opción solo si el kit de un clic no funciona, o si necesita instalarlo en Linux o macOS. Aquí sí hay que instalar PostgreSQL aparte.

Requisitos adicionales

PostgreSQL 14 o superior, corriendo en el puerto normal **5432**. Debe conocer la contraseña del usuario `postgres`.

Paso 1 — Crear las bases de datos

Con pgAdmin o con `psql`, ejecute:

```
CREATE DATABASE app_iunav;
CREATE DATABASE app_iunav_test;
```

La segunda es opcional: solo se usa para las pruebas automáticas del proyecto.

Paso 2 — Configurar la API

Dentro de **`server\`**, copie el archivo de ejemplo y edítelo:

```
cd server
copy .env.example .env        :: Windows CMD
# En PowerShell o Linux/macOS:  cp .env.example .env
```

Los valores mínimos que debe revisar en **`server\.env`**:

```
DATABASE_URL=postgresql://postgres:TU_CONTRASENA@localhost:5432/app_iunav
DB_SSL=false
JWT_SECRET=una-clave-larga-y-aleatoria-de-al-menos-32-caracteres
FRONTEND_URL=http://localhost:3000
PORT=4000
API_HOST=127.0.0.1
```

> **DB_SSL debe quedar en `false`**
>
> En local no hay cifrado. Si se deja en `true`, la conexión a la base falla.

Paso 3 — Instalar, crear el esquema y cargar los datos

```
cd server
npm ci
npm run db:schema
npm run db:seed -- --confirm-seed
```

Paso 4 — Levantar la API (primera terminal)

```
cd server
npm run dev
```

Debe quedar escuchando en `http://localhost:4000`.

Paso 5 — Levantar la web (segunda terminal)

```
cd web
npm ci
npm run dev
```

Abra `http://localhost:3000`. Si la web no encuentra la API, cree el archivo **`web\.env.local`** con una sola línea:

```
NEXT_PUBLIC_API_URL=http://localhost:4000
```

Paso 6 — Detener

En cada terminal, pulse `Ctrl` + `C`. Para apagar PostgreSQL, use el panel del servicio o `pg_ctl stop`.

> **Diferencias con la Opción A**
>
> En la Opción B usted administra PostgreSQL. En la Opción A el proyecto lo hace todo: no hay que crear bases ni editar `server\.env`.

12. Cómo desinstalar y dejar el equipo limpio

El proyecto no instala nada fuera de su carpeta: no toca el registro de Windows, no crea servicios y no deja programas residentes. Para eliminarlo:

1. Ejecute `detener.cmd` para apagar todo.
2. Borre la carpeta completa del proyecto.
3. Opcional: desinstale Node.js desde *Configuración → Aplicaciones* si no lo usa para otra cosa.

La base de datos vive dentro de **`local\.pgdata\`**, así que se borra junto con la carpeta. No queda nada suelto en el sistema.

> **Puede mover el proyecto de carpeta**
>
> Si lo copia completo (incluyendo **`local\.pgdata\`** y **`local\.env.local`**), sigue funcionando. Solo no lo mueva mientras esté en marcha.

13. Anexo: estructura y comandos

Qué hay en cada carpeta

```
servicio-comunitario-iunav\
├── local\
│   ├── instalar.cmd    ← prepara todo (una sola vez)
│   ├── iniciar.cmd     ← arranca el sistema
│   ├── detener.cmd     ← lo apaga
│   ├── app.mjs         ← lanzador de la API y la web
│   ├── pg.mjs          ← control del PostgreSQL incluido
│   ├── .pgdata\        ← la base de datos (se crea al instalar)
│   ├── .env.local      ← contraseñas locales (se crea al instalar)
│   └── logs\           ← registros de cada servicio
├── server\             ← la API (Express + PostgreSQL)
│   ├── sql\schema.sql  ← las 9 tablas
│   └── src\            ← el código
├── web\                ← la web (Next.js)
│   └── src\            ← pantallas y componentes
├── deploy\             ← scripts para publicar en un servidor (no se usan aquí)
└── docs\               ← este manual y las guías de usuario
```

Comandos útiles

Desde la carpeta **`local\`**:

Comando	Qué hace
`node app.mjs status`	Dice si la API y la web están corriendo y si la base responde.
`node app.mjs start`	Arranca API y web (lo que hace `iniciar.cmd`).
`node app.mjs stop`	Cierra API y web.
`node pg.mjs status`	Estado de la base de datos local.
`node pg.mjs stop`	Apaga la base de datos.

Desde la carpeta **`server\`**:

Comando	Qué hace
`npm run db:schema`	Crea o actualiza las tablas.
`npm run db:seed -- --confirm-seed`	Carga los datos de ejemplo.
`npm run db:wipe`	Borra los datos y deja un administrador.
`npm test`	Ejecuta las 449 pruebas automáticas del proyecto.

Documentación complementaria

En la carpeta **`docs\`** del proyecto:

Archivo	Contenido
`Guia-de-Usuario-IUNAV.pdf`	Qué puede hacer cada rol, paso a paso.
`Flujo-de-Uso-IUNAV.pdf`	El ciclo completo del programa, por etapas.
`Manual-Despliegue-Local.pdf`	Este documento.

Ficha técnica

Aspecto	Detalle
Lenguaje	TypeScript
API	Node.js + Express 4
Web	Next.js 16 + React 19 + Tailwind 4
Base de datos	PostgreSQL 17
Autenticación	JWT con 4 roles y permisos por endpoint
Pruebas	449 pruebas automáticas (Vitest + Supertest)
Endpoints	48 rutas con su matriz de permisos
